



Heaven's Light is Our Guide

Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology, Bangladesh

Course Code: CSE 2203
Course Title: Digital Techniques

Presented by,
Prof. Dr. Boshir Ahmed

Canonical and Standard Forms

- We need to consider formal techniques for the simplification of Boolean functions.
 - Identical functions will have exactly the same canonical form.
 - Minterms and Maxterms
 - Sum-of-Minterms and Product-of- Maxterms
 - Product and Sum terms
 - Sum-of-Products (SOP) and Product-of-Sums (POS)

Definitions

- *Literal*: A variable or its complement
- *Product term*: literals connected by $\cdot$
- *Sum term*: literals connected by $+$
- *Minterm*: a product term in which all the variables appear exactly once, either complemented or uncomplemented
- *Maxterm*: a sum term in which all the variables appear exactly once, either complemented or uncomplemented

Minterm

- Represents exactly one combination in the truth table.
- Denoted by m_j , where j is the decimal equivalent of the minterm's corresponding binary combination (b_j).
- A variable in m_j is complemented if its value in b_j is 0, otherwise is uncomplemented.
- Example: Assume 3 variables (A,B,C), and $j=3$. Then, $b_j = 011$ and its corresponding minterm is denoted by $m_j = A'BC$

Maxterm

- Represents exactly one combination in the truth table.
- Denoted by M_j , where j is the decimal equivalent of the maxterm's corresponding binary combination (b_j).
- A variable in M_j is complemented if its value in b_j is 1, otherwise is uncomplemented.
- Example: Assume 3 variables (A,B,C), and $j=3$. Then, $b_j = 011$ and its corresponding maxterm is denoted by $M_j = A+B'+C'$

Truth Table notation for Minterms and Maxterms

- Minterms and Maxterms are easy to denote using a truth table.
- Example:
Assume 3 variables x, y, z (order is fixed)

x	y	z		Minterm	Maxterm
0	0	0		$x'y'z' = m_0$	$x+y+z = M_0$
0	0	1		$x'y'z = m_1$	$x+y+z' = M_1$
0	1	0		$x'yz' = m_2$	$x+y'+z = M_2$
0	1	1		$x'yz = m_3$	$x+y'+z' = M_3$
1	0	0		$xy'z' = m_4$	$x'+y+z = M_4$
1	0	1		$xy'z = m_5$	$x'+y+z' = M_5$
1	1	0		$xyz' = m_6$	$x'+y'+z = M_6$
1	1	1		$xyz = m_7$	$x'+y'+z' = M_7$

Canonical Forms (Unique)

- Any Boolean function $F()$ can be expressed as a *unique* **sum** of **minterms** and a unique **product** of **maxterms** (under a fixed variable ordering).
- In other words, every function $F()$ has two canonical forms:
 - Canonical Sum-Of-Products (sum of minterms)
 - Canonical Product-Of-Sums (product of maxterms)

Canonical Forms (cont.)

- Canonical Sum-Of-Products:
The minterms included are those m_j such that $F(\) = 1$ in row j of the truth table for $F(\)$.
- Canonical Product-Of-Sums:
The maxterms included are those M_j such that $F(\) = 0$ in row j of the truth table for $F(\)$.

Example

- Truth table for $f_1(a,b,c)$ at right
- The canonical sum-of-products form for f_1 is
$$f_1(a,b,c) = m_1 + m_2 + m_4 + m_6$$
$$= a'b'c + a'bc' + ab'c' + abc'$$
- The canonical product-of-sums form for f_1 is
$$f_1(a,b,c) = M_0 \cdot M_3 \cdot M_5 \cdot M_7$$
$$= (a+b+c) \cdot (a+b'+c') \cdot (a'+b+c') \cdot (a'+b'+c').$$
- Observe that: $m_j = M_j'$

a	b	c	f_1
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Shorthand: $\sum$ and $\prod$

- $f_1(a,b,c) = \sum m(1,2,4,6)$, where $\sum$ indicates that this is a sum-of-products form, and $m(1,2,4,6)$ indicates that the minterms to be included are m_1 , m_2 , m_4 , and m_6 .
- $f_1(a,b,c) = \prod M(0,3,5,7)$, where $\prod$ indicates that this is a product-of-sums form, and $M(0,3,5,7)$ indicates that the maxterms to be included are M_0 , M_3 , M_5 , and M_7 .
- Since $m_j = M_j'$ for any j ,
$$\sum m(1,2,4,6) = \prod M(0,3,5,7) = f_1(a,b,c)$$

Conversion Between Canonical Forms

- Replace $\sum$ with $\prod$ (or *vice versa*) and replace those j 's that appeared in the original form with those that do not.

- Example:

$$\begin{aligned}f_1(a,b,c) &= a'b'c + a'bc' + ab'c' + abc' \\&= m_1 + m_2 + m_4 + m_6 \\&= \sum(1,2,4,6) \\&= \prod(0,3,5,7) \\&= (a+b+c) \cdot (a+b'+c') \cdot (a'+b+c') \cdot (a'+b'+c')\end{aligned}$$

Standard Forms (NOT Unique)

- Standard forms are “*like*” canonical forms, except that not all variables need appear in the individual product (SOP) or sum (POS) terms.
- Example:
$$f_1(a,b,c) = a'b'c + bc' + ac'$$

is a *standard* sum-of-products form
- $$f_1(a,b,c) = (a+b+c) \cdot (b'+c') \cdot (a'+c')$$

is a *standard* product-of-sums form.

Conversion of SOP from standard to canonical form

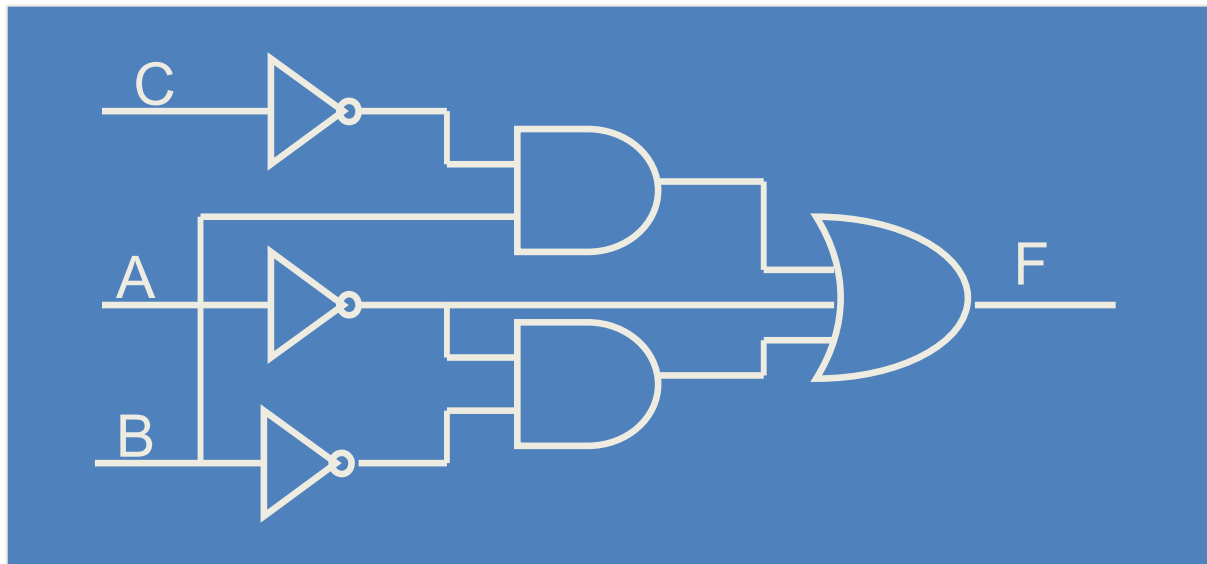
- Expand *non-canonical* terms by inserting equivalent of 1 in each missing variable x:
 $(x + x') = 1$
- Remove duplicate minterms
- $f_1(a,b,c) = a'b'c + bc' + ac'$
 $= a'b'c + (a+a')bc' + a(b+b')c'$
 $= a'b'c + abc' + a'bc' + abc' + ab'c'$
 $= a'b'c + abc' + a'bc + ab'c'$

Conversion of POS from standard to canonical form

- Expand noncanonical terms by adding 0 in terms of missing variables (e.g., $xx' = 0$) and using the distributive law
- Remove duplicate maxterms
- $$\begin{aligned} f_1(a,b,c) &= (a+b+c) \cdot (b'+c') \cdot (a'+c') \\ &= (a+b+c) \cdot (\textcolor{red}{a}a' + b' + c') \cdot (a' + \textcolor{red}{b}b' + c') \\ &= (a+b+c) \cdot (a+b'+c') \cdot (\textcolor{blue}{a'} + b' + \textcolor{blue}{c'}) \cdot \\ &\quad (a' + b + c') \cdot (\textcolor{blue}{a'} + \textcolor{blue}{b'} + \textcolor{blue}{c'}) \\ &= (a+b+c) \cdot (a+b'+c') \cdot (a'+b'+c') \cdot (a'+b+c') \end{aligned}$$

Combinational Logic Circuit from Logic Function

- Consider function $F = A' + B \cdot C' + A' \cdot B'$
- A combinational logic circuit can be constructed to implement F , by appropriately connecting input signals and logic gates:
 - Circuit input signals $\rightarrow$ from function variables (A, B, C)
 - Circuit output signal $\rightarrow$ function output (F)
 - Logic gates $\rightarrow$ from logic operations

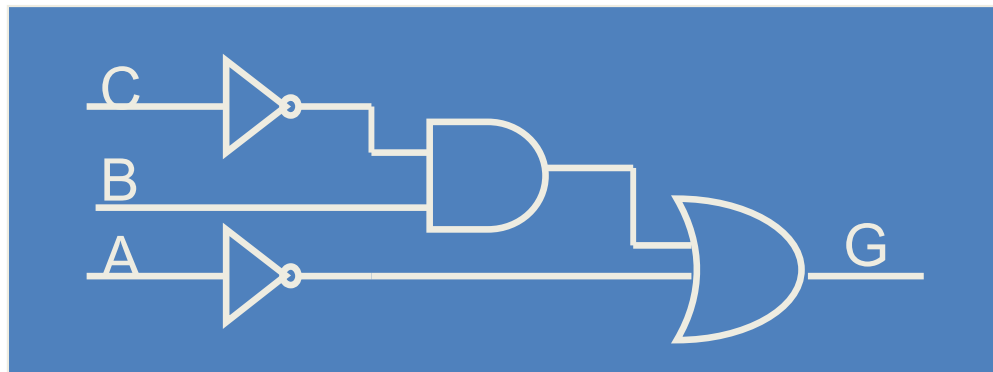
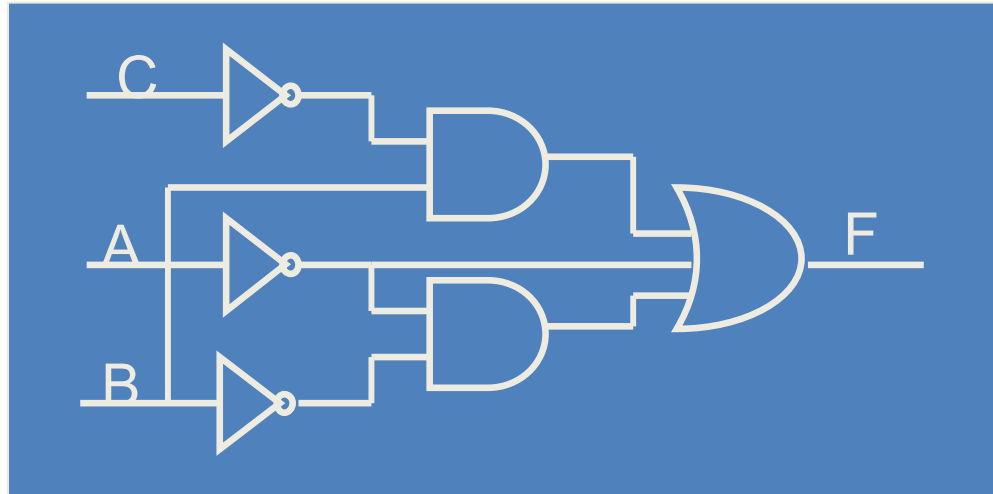


Combinational Logic Circuit from Logic Function (cont.)

- In order to design a cost-effective and efficient circuit, we must minimize the circuit's size (area) and propagation delay (time required for an input signal change to be observed at the output line)
- Observe the truth table of $F=A' + B \cdot C'$ + $A' \cdot B'$ and $G=A' + B \cdot C'$
- Truth tables for F and G are identical
→ same function
- Use G to implement the logic circuit (less components)

A	B	C	F	G
0	0	0	1	1
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	0	0

Combinational Logic Circuit from Logic Function (cont.)



Circuit Simplification Methods

- Boolean algebra: greatly depends on inspiration and experience.
- Karnaugh map: systematic, step-by-step approach.

Algebraic Simplification

- Use the Boolean algebra theorems introduced in Lecture 4 to help simplify the expression for a logic circuit.
- Based on experience, often becomes a trial-and-error process.
- No easy way to tell whether a simplified expression is in its simplest form.

Two Essential Steps

- The original expression is put into the sum-of-products form by repeated application of DeMorgan's theorem and multiplication of terms.
- The product terms are checked for common factors, and factoring is performed whenever possible.

Examples 1- 4

Original	Simplified
$ABC + AB'(A'C)'$	$A(B' + C)$
$ABC + ABC' + AB'C$	$A(B + C)$
$A'C(A'BD)' + A'BC'D' + AB'C$	$B'C + A'D'(B + C)$
$(A' + B)(A + B + D)D'$	BD'

Examples 5 - 6

- $(A'+B)(A+B')$: equivalent form $A'B'+AB$
- $AB'C+A'BD+C'D'$: cannot be simplified further.

Boolean expressions-NOT unique

- Unlike truth tables, expressions representing a Boolean function are NOT unique.
- Example:
 - $F(x,y,z) = x' \cdot y' \cdot z' + x' \cdot y \cdot z' + x \cdot y \cdot z'$
 - $G(x,y,z) = x' \cdot y' \cdot z' + y \cdot z'$
- The corresponding truth tables for $F()$ and $G()$ are to the right. They are identical.
- Thus, $F() = G()$

x	y	z		F	G
0	0	0		1	1
0	0	1		0	0
0	1	0		1	1
0	1	1		0	0
1	0	0		0	0
1	0	1		0	0
1	1	0		1	1
1	1	1		0	0

Algebraic Manipulation

Example 7

- Boolean algebra is a useful tool for simplifying digital circuits.
- Why do it? Simpler can mean cheaper, smaller, faster.
- Example: Simplify $F = x'yz + x'yz' + xz$.

$$\begin{aligned} F &= x'yz + x'yz' + xz \\ &= x'y(z+z') + xz \\ &= x'y \cdot 1 + xz \\ &= x'y + xz \end{aligned}$$

Algebraic Manipulation (cont.)

Example 8

- Example: Prove

$$x'y'z' + x'yz' + xyz' = x'z' + yz'$$

- **Proof:**

$$\begin{aligned} x'y'z' + x'yz' + xyz' \\ &= x'y'z' + x'yz' + x'yz' + xyz' \\ &= x'z'(y' + y) + yz'(x' + x) \\ &= x'z' \cdot 1 + yz' \cdot 1 \\ &= x'z' + yz' \end{aligned}$$

Complement of a Function

- The complement of a function is derived by interchanging ($\cdot$ and $+$), and (1 and 0), and complementing each variable.
- Otherwise, interchange 1s to 0s in the truth table column showing F.
- The *complement* of a function IS NOT THE SAME as the *dual* of a function.

Complementation

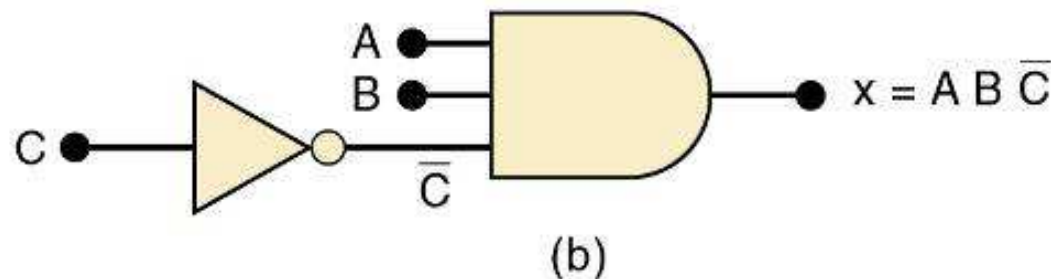
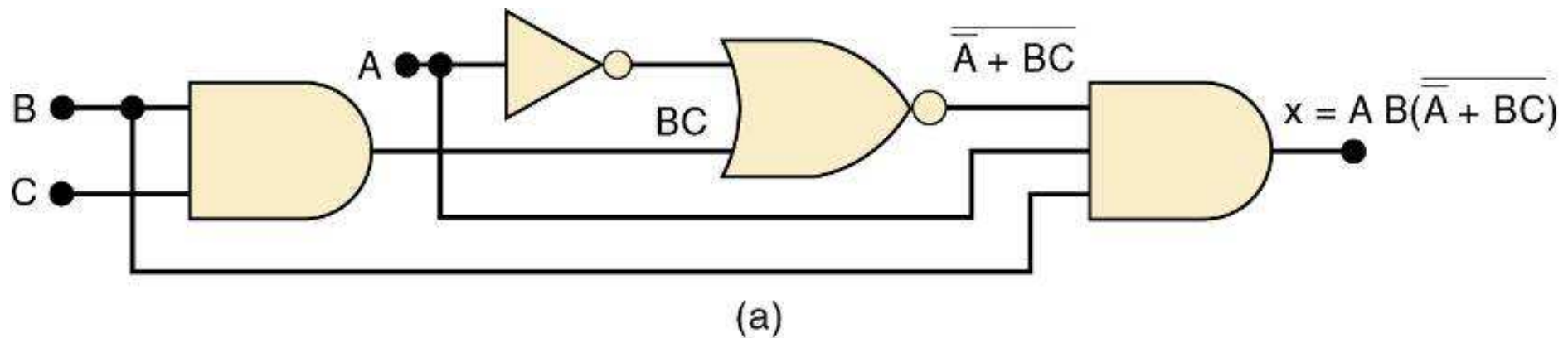
Example 9

- Find $G(x,y,z)$, the complement of $F(x,y,z) = xy'z' + x'yz$
- $G = F' = (xy'z' + x'yz)'$
 $= (xy'z')' \cdot (x'yz)'$ *DeMorgan*
 $= (x' + y + z) \cdot (x + y' + z')$ *DeMorgan* again
- Note: The complement of a function can also be derived by finding the function's *dual*, and then complementing all of the literals

Simplifying Logic Circuits (Practice)

Example 10

- The circuits shown provide the same output
 - Circuit (b) is clearly less complex.

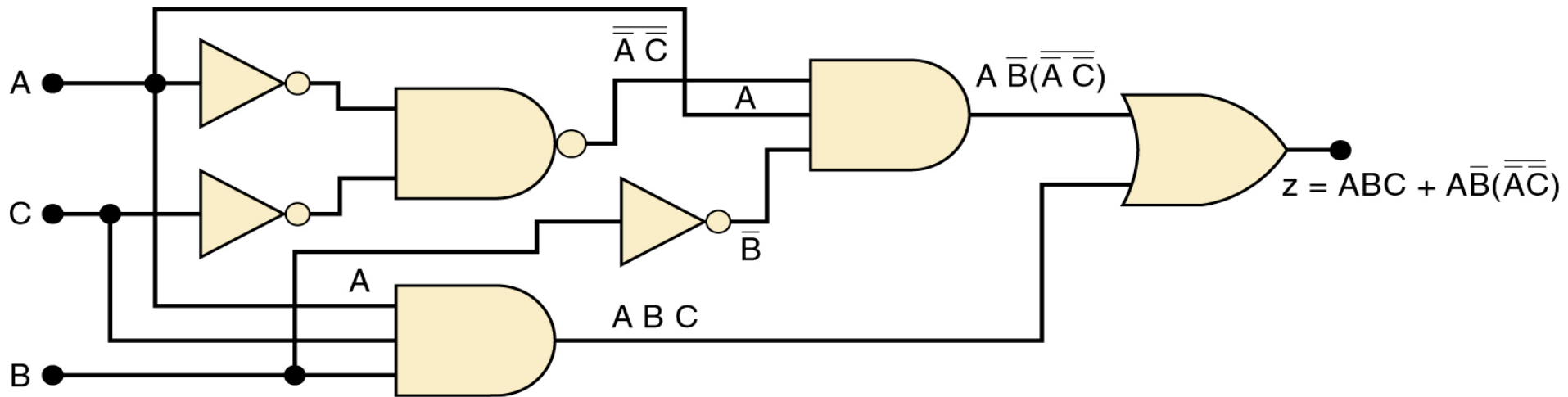


Logic circuits can be simplified using Boolean algebra and Karnaugh mapping.

Algebraic Simplification

Example 11

Simplify the logic circuit shown.



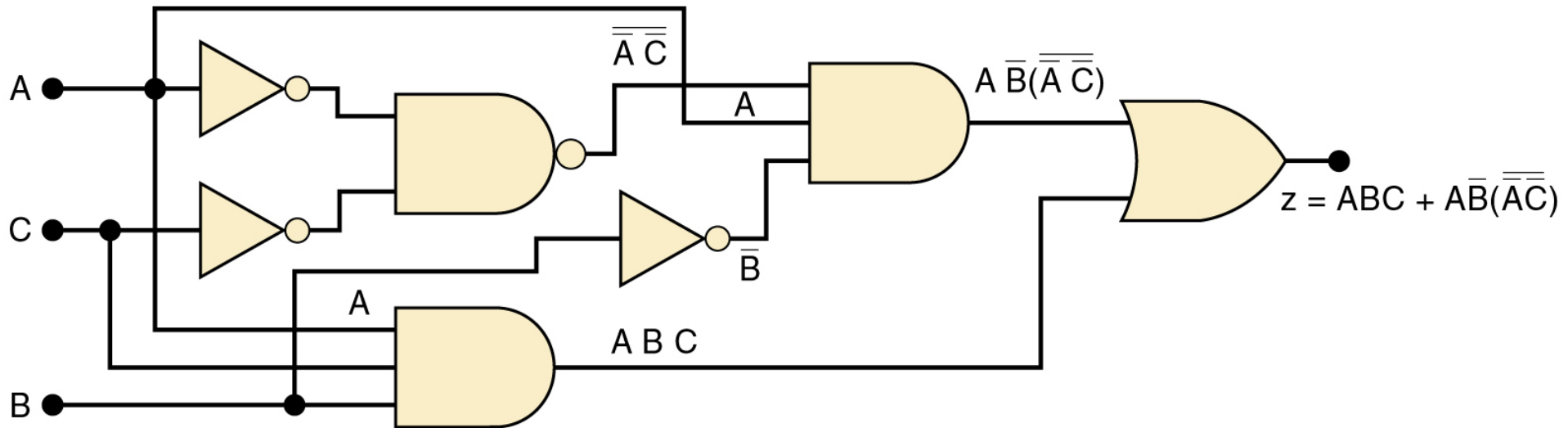
The first step is to determine the expression for the output: $z = ABC + AB\bar{A}\bar{C}$

Once the expression is determined, break down large inverter signs by DeMorgan's theorems & multiply out all terms.

$$\begin{aligned} z &= ABC + AB\bar{A}\bar{C} && [\text{theorem (17)}] \\ &= ABC + AB\bar{(A + C)} && [\text{cancel double inversions}] \\ &= ABC + AB\bar{A}\bar{A} + AB\bar{C}\bar{C} && [\text{multiply out}] \\ &= ABC + AB\bar{A} + AB\bar{C} && [A \cdot A = A] \end{aligned}$$

Algebraic Simplification

Simplify the logic circuit shown.



Factoring—the first & third terms above have **AC** in common, which can be factored out:

$$z = AC(B + \overline{B}) + A\overline{B}$$

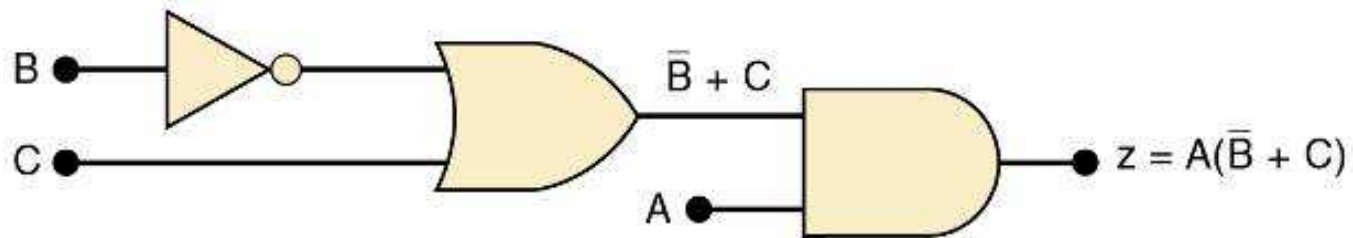
Since $B + \overline{B} = 1$, then...

$$\begin{aligned} z &= AC(1) + A\overline{B} \\ &= AC + A\overline{B} \end{aligned}$$

Factor out **A**, which results in...

Algebraic Simplification

Simplified logic circuit.



$$z = A(C + \bar{B})$$

Karnaugh Maps

- Karnaugh maps (K-maps) are *graphical* representations of Boolean functions.
- One ***map cell*** corresponds to a row in the truth table.
- Also, one map cell corresponds to a minterm or a maxterm in the Boolean expression
- Multiple-cell areas of the map correspond to standard terms.

Karnaugh Map Method

- A graphical device to simplify a logic expression.
- Generally work on examples with up to 4 input variables.
- From truth table to logic expression to K map.

Complete Simplification Process

- Step 1: Construct the K map and places 1s in those squares corresponding to the 1s in the truth table. Places 0s in the other squares.
- Step 2: Examine the map for adjacent 1s and loop those 1s which are not adjacent to any other 1s. (isolated 1s)
- Step 3: Look for those 1s which are adjacent to only one other 1. Loop any pair containing such a 1.
- Step 4: Loop any octet even when it contains some 1s that have already been looped.

Complete Simplification Process

- Step 5: Loop any quad that contains one or more 1s that have not already been looped, making sure to use the minimum number of loops.
- Step 6: Loop any pairs necessary to include any 1s have not already been looped, making sure to use the minimum number of loops.
- Step 7: Form the ORed sum of all the terms generated by each loop.

Filling K Map from Output Expression

- What to do when the desired output is presented as a Boolean expression instead of a truth table?
- Step 1: Convert the expression into SOP form.
- Step 2: For each product term in the SOP expression, place a 1 in each K-map square whose label contains the same combination of input values. Place a 0 in other squares.
- Example 9: $y = C'(A'B'D' + D) + AB'C + D'$

Two-Variable Map

$x_1 \backslash x_2$		0	1
0	0 m_0	1 m_1	
1	2 m_2	3 m_3	

OR

x_1		0	1
x_2	0	0 m_0	2 m_2
	1	1 m_1	3 m_3

NOTE: ordering of variables is IMPORTANT for $f(x_1, x_2)$, x_1 is the row, x_2 is the column.

Cell 0 represents $x_1'x_2'$; Cell 1 represents $x_1'x_2$; etc. If a minterm is present in the function, then a 1 is placed in the corresponding cell.

Two-Variable Map (cont.)

- Any two adjacent cells in the map differ by ONLY one variable, which appears complemented in one cell and uncomplemented in the other.
- Example:
 $m_0 (=x_1'x_2')$ is adjacent to $m_1 (=x_1'x_2)$ and $m_2 (=x_1x_2')$ but NOT $m_3 (=x_1x_2)$

2-Variable Map -- Example

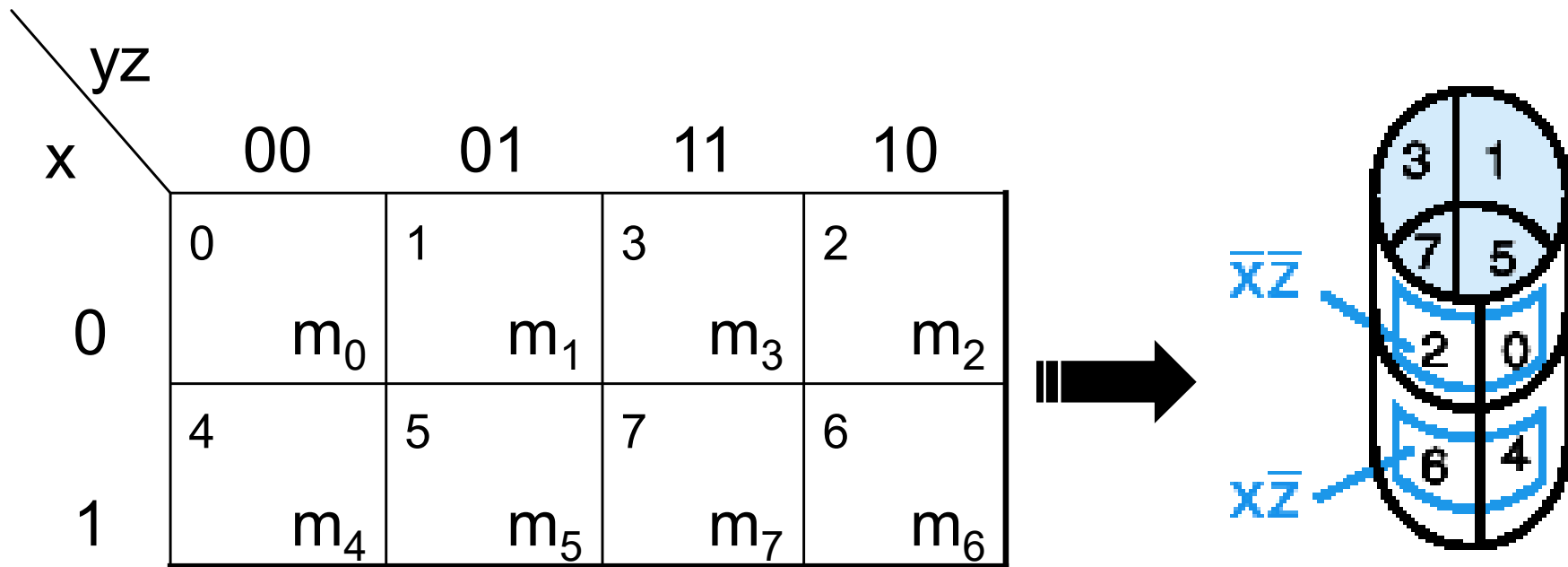
- $f(x_1, x_2) = x_1'x_2' + x_1'x_2 + x_1x_2'$
 $= m_0 + m_1 + m_2$
 $= x_1' + x_2'$
- 1s placed in K-map for specified minterms m_0, m_1, m_2
- Grouping (ORing) of 1s allows simplification
- What (simpler) function is represented by each dashed rectangle?
 - $x_1' = m_0 + m_1$
 - $x_2' = m_0 + m_2$
- Note m_0 covered twice

		x_2	
		0	1
x_1	0	0	1
	1	2	3
0		1	1
1		1	0

Minimization as SOP using K-map

- Enter 1s in the K-map for each product term in the function
- Group *adjacent* K-map cells containing 1s to obtain a product with fewer variables. Group size must be in power of 2 (2, 4, 8, ...)
- Handle “boundary wrap” for K-maps of 3 or more variables.
- Realize that answer may not be unique

Three-Variable Map



- Note: variable ordering is (x,y,z); yz specifies column, x specifies row.
- Each cell is adjacent to **three** other cells (left or right or top or bottom or edge wrap)

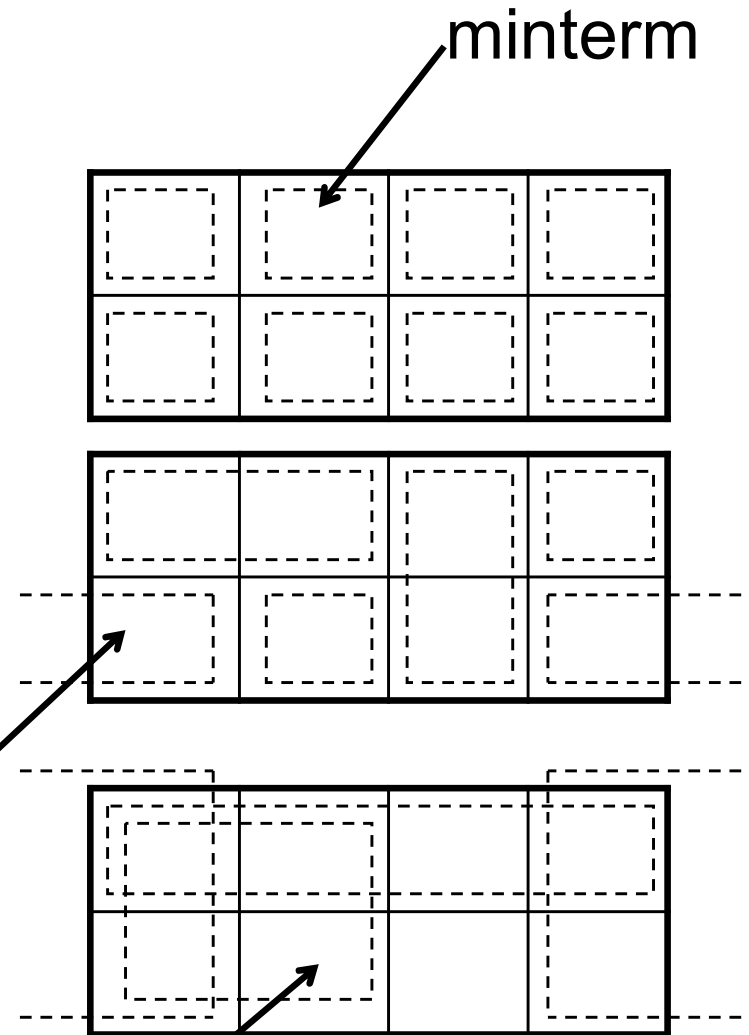
Three-Variable Map (cont.)

The types of structures that are either minterms or are generated by repeated application of the minimization theorem on a three variable map are shown at right.

Groups of 1, 2, 4, 8 are possible.

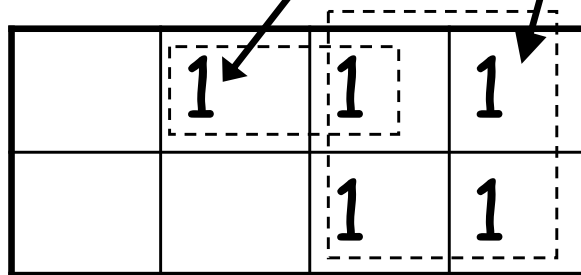
group of 2 terms

group of 4 terms



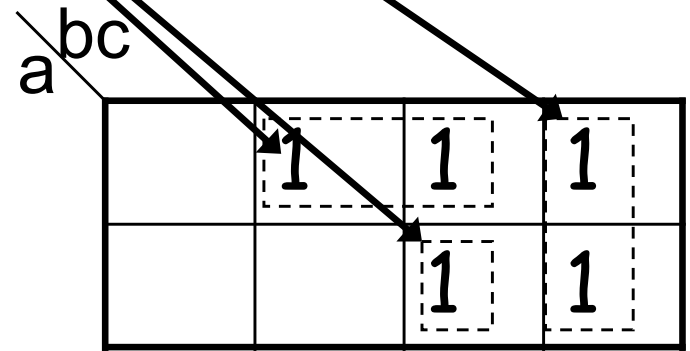
Simplification

- Enter minterms of the Boolean function into the map, then group terms
- Example: $f(a,b,c) = a'c + abc + bc'$
- Result: $f(a,b,c) = a'c + b$



A 2x4 Karnaugh map for variables a, b, and c. The top row represents a=0 and the bottom row represents a=1. The columns represent bc values of 00, 01, 11, and 10. The map contains 1s at positions (0,01), (0,11), (0,10), (1,11), and (1,10). Two dashed boxes indicate groupings: one for the top row (a'c) and one for the last two columns (b).

	1	1	1
		1	1



A 2x4 Karnaugh map for variables a, b, and c. The top row represents a=0 and the bottom row represents a=1. The columns represent bc values of 00, 01, 11, and 10. The map contains 1s at positions (0,01), (0,11), (0,10), (1,11), and (1,10). Three dashed boxes indicate groupings: one for the top row (a'c), one for the intersection of a=0 and bc=11 (abc), and one for the intersection of a=0 and bc=10 (bc').

	1	1	1
		1	1

More Examples

- $f_1(x, y, z) = \sum m(2,3,5,7)$

- $f_1(x, y, z) = x'y + xz$

		yz			
		00	01	11	10
x	0			1	1
	1		1	1	

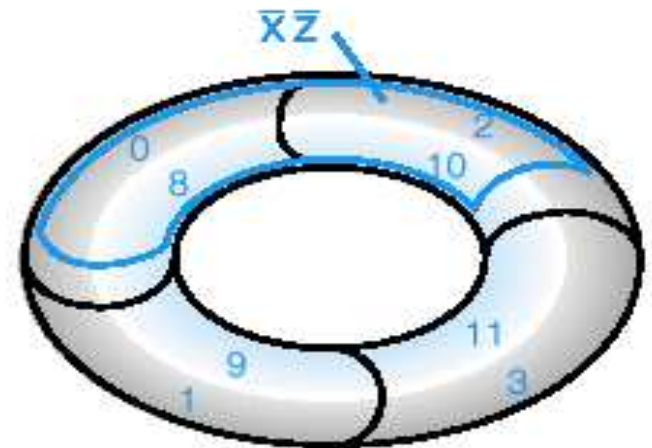
- $f_2(x, y, z) = \sum m(0,1,2,3,6)$

- $f_2(x, y, z) = x' + yz'$

1	1	1	1
			1

Four-Variable Maps

		YZ			
		00	01	11	10
WX	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}



- Top cells are adjacent to bottom cells. Left-edge cells are adjacent to right-edge cells.
- Note variable ordering (WXYZ).

Four-variable Map Simplification

- One square represents a minterm of 4 literals.
- A rectangle of 2 adjacent squares represents a product term of 3 literals.
- A rectangle of 4 squares represents a product term of 2 literals.
- A rectangle of 8 squares represents a product term of 1 literal.
- A rectangle of 16 squares produces a function that is equal to logic 1.

Example

- Simplify the following Boolean function $(A,B,C,D) = \sum m(0,1,2,4,5,7,8,9,10,12,13)$.
- First put the function $g()$ into the map, and then group as many 1s as possible.

cd

ab

1	1		1
1	1	1	
1	1		
1	1		1

1	1		1
1	1	1	
1	1		
1	1		1

$$g(A,B,C,D) = c' + b'd' + a'bd$$

Don't Care Conditions

- There may be a combination of input values which
 - will **never** occur
 - if they do occur, the output is of no concern.
- The function value for such combinations is called a *don't care*.
- They are denoted with **x** or **—**. Each **x** may be arbitrarily assigned the value 0 or 1 in an implementation.
- Don't cares can be used to **further** simplify a function

Don't-Care Conditions

- Some logic circuits can be designed so that there are certain input conditions for which there are no specified output levels.
- A circuit designer is free to make the output for any don't care condition either a 0 or a 1 in order to produce the simplest output expression.

Example

- Simplify the function $f(a,b,c,d)$ whose K-map is shown at the right.
- $f = a'c'd + ab' + cd' + a'bc'$
or
- $f = a'c'd + ab' + cd' + a'bd'$

ab \ cd	cd			
	00	01	11	10
00	0	1	0	1
01	1	1	0	1
11	0	0	x	x
10	1	1	x	x

0	1	0	1
1	1	0	1
0	0	x	x
1	1	x	x

0	1	0	1
1	1	0	1
0	0	x	x
1	1	x	x

Another Example

- Simplify the function $g(a,b,c,d)$ whose K-map is shown at right.
- $g = a'c' + ab$
or
- $g = a'c' + b'd$

ab \ cd					
		00	01	11	10
00	x	1	0	0	0
01	1	x	0	x	
11	1	x	x	1	
10	0	x	x	0	

ab \ cd					
		00	01	11	10
00	x	1	0	0	0
01	1	x	0	x	
11	1	x	x	1	
10	0	x	x	0	

ab \ cd					
		00	01	11	10
00	x	1	0	0	0
01	1	x	0	x	
11	1	x	x	1	
10	0	x	x	0	

Another Example

For each symbol of the Excess-3 code, we use 1's to draw the map for simplifying Boolean function.

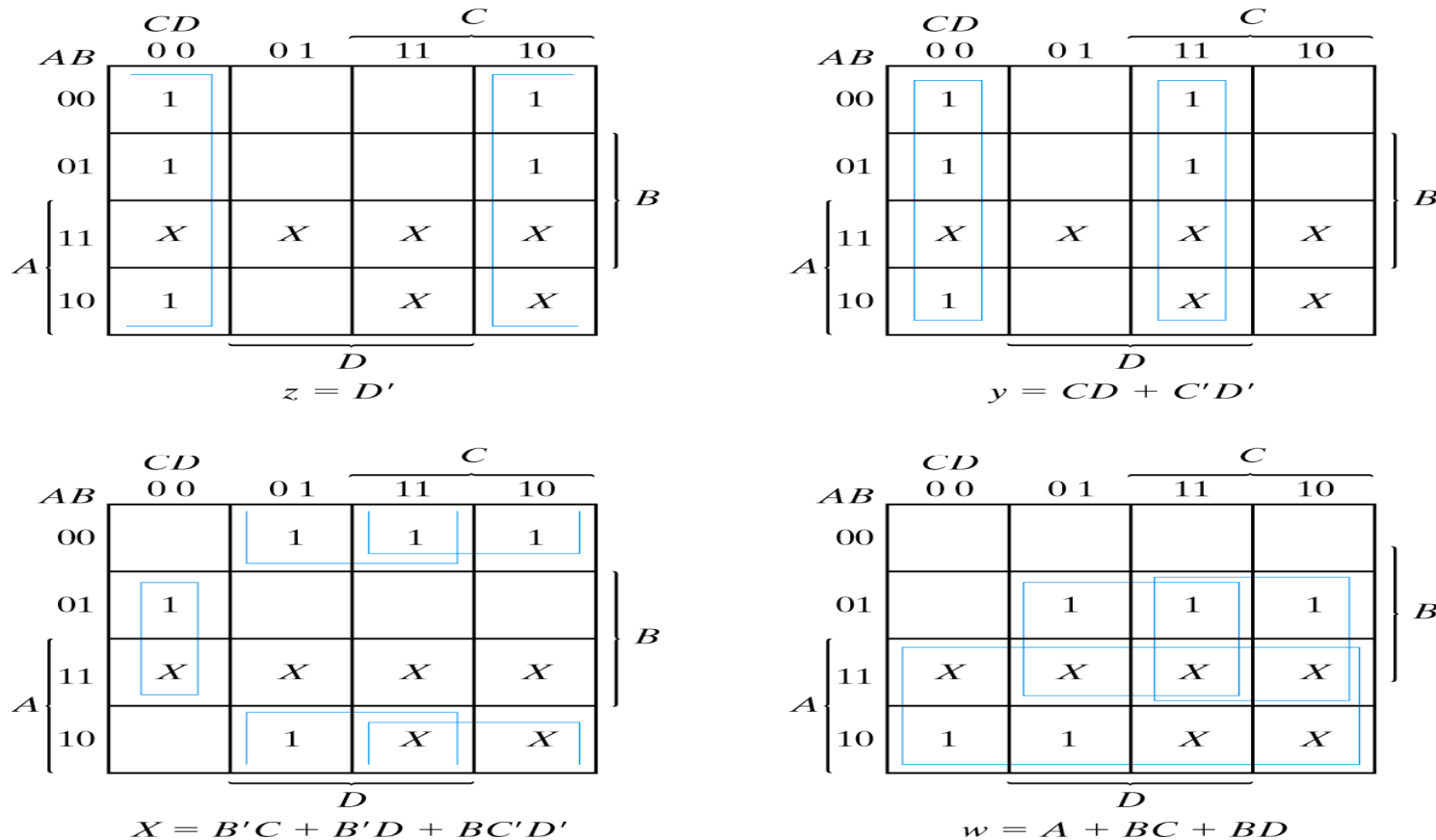


Fig. 4-3 Maps for BCD to Excess-3 Code Converter

Circuit implementation

$$z = D'; \quad y = CD + C'D' = CD + (C + D)'$$

$$x = B'C + B'D + BC'D' = B'(C + D) + B(C + D)'$$

$$w = A + BC + BD = A + B(C + D)$$

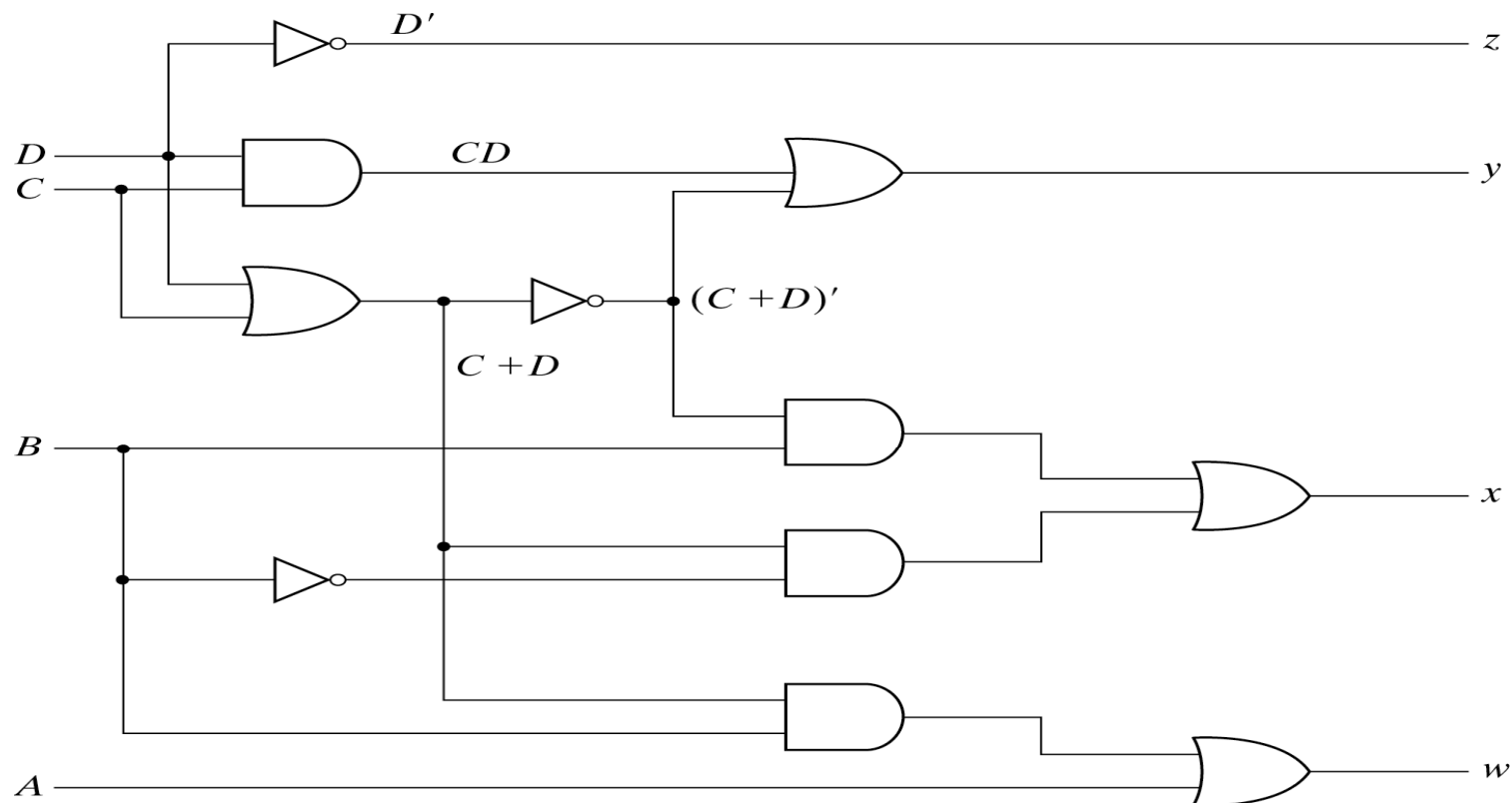
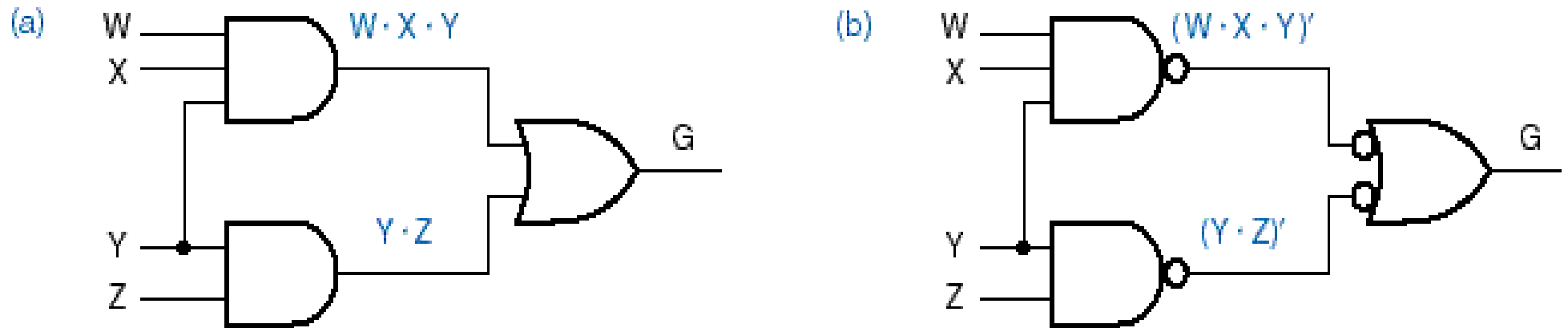


Fig. 4-4 Logic Diagram for BCD to Excess-3 Code Converter

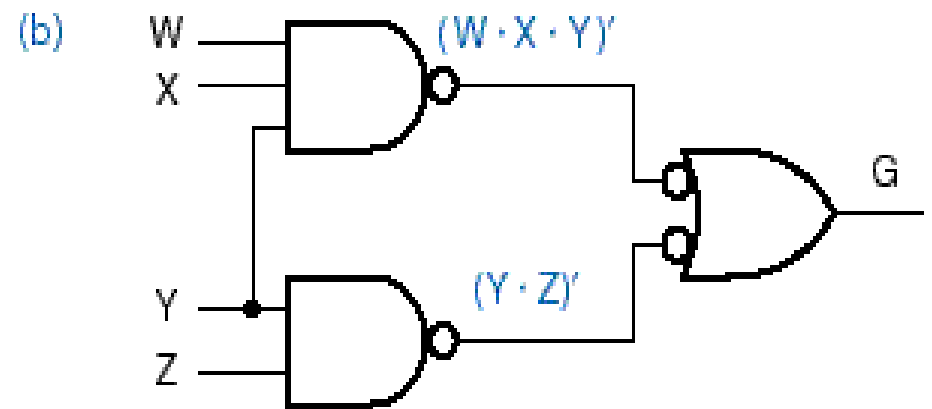
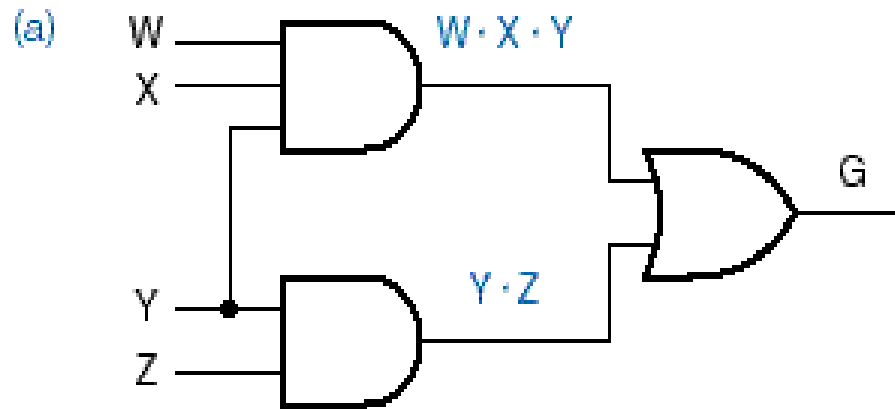
AND-OR (SOP) Emulation Using NANDs



Two-level implementations

- a) Original SOP
- b) Implementation with NANDs

AND-OR (SOP) Emulation Using NANDs (cont.)

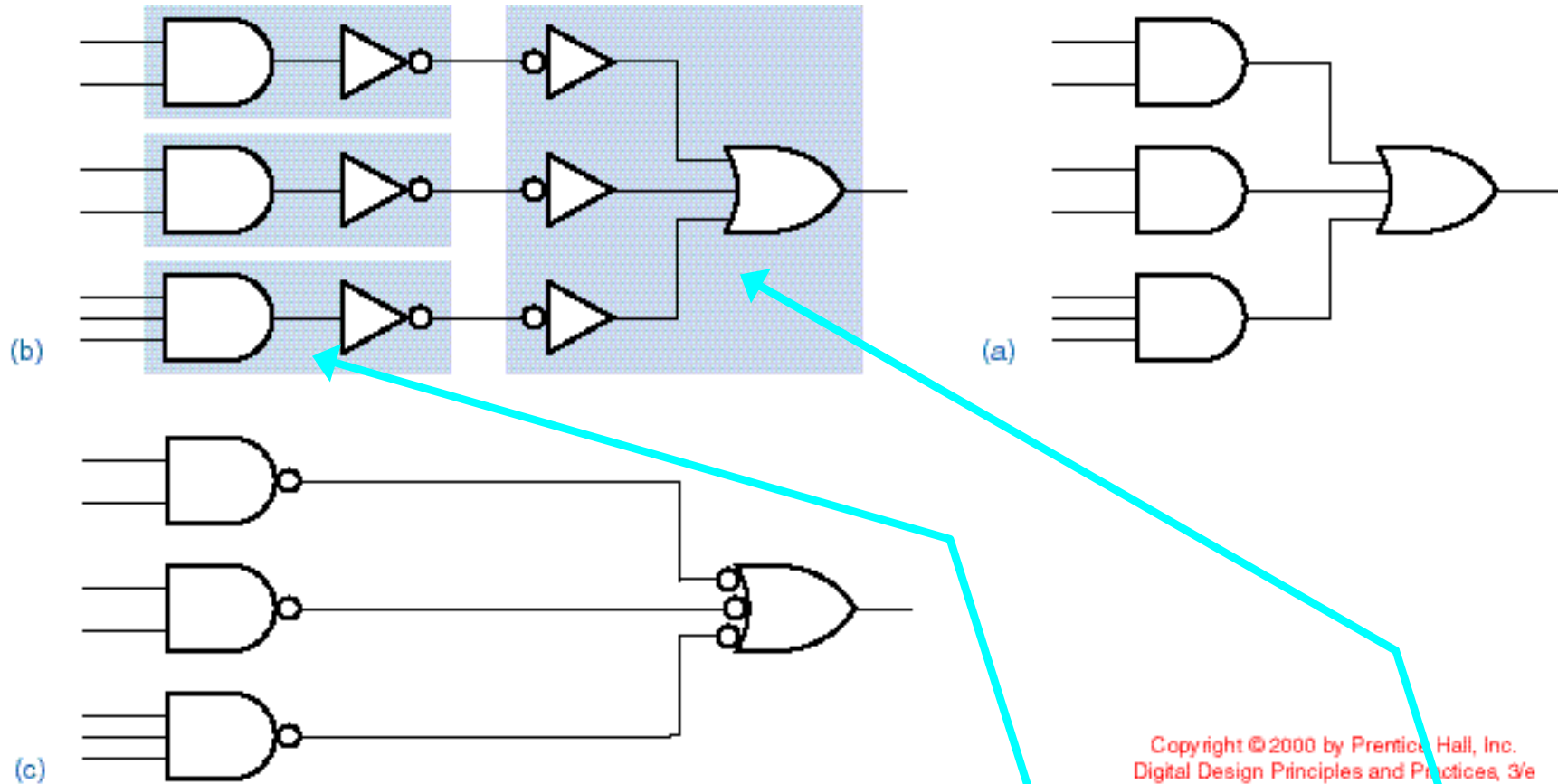


Verify:

(a) $G = WXY + YZ$

(b) $G = ((WXY)' \cdot (YZ)')'$
 $= (WXY)'' + (YZ)'' = WXY + YZ$

SOP with NAND



- (a) Original SOP
- (b) Double inversion and grouping
- (c) Replacement with NANDs

AND-NOT

NOT-OR

Two-Level NAND Gate Implementation - Example

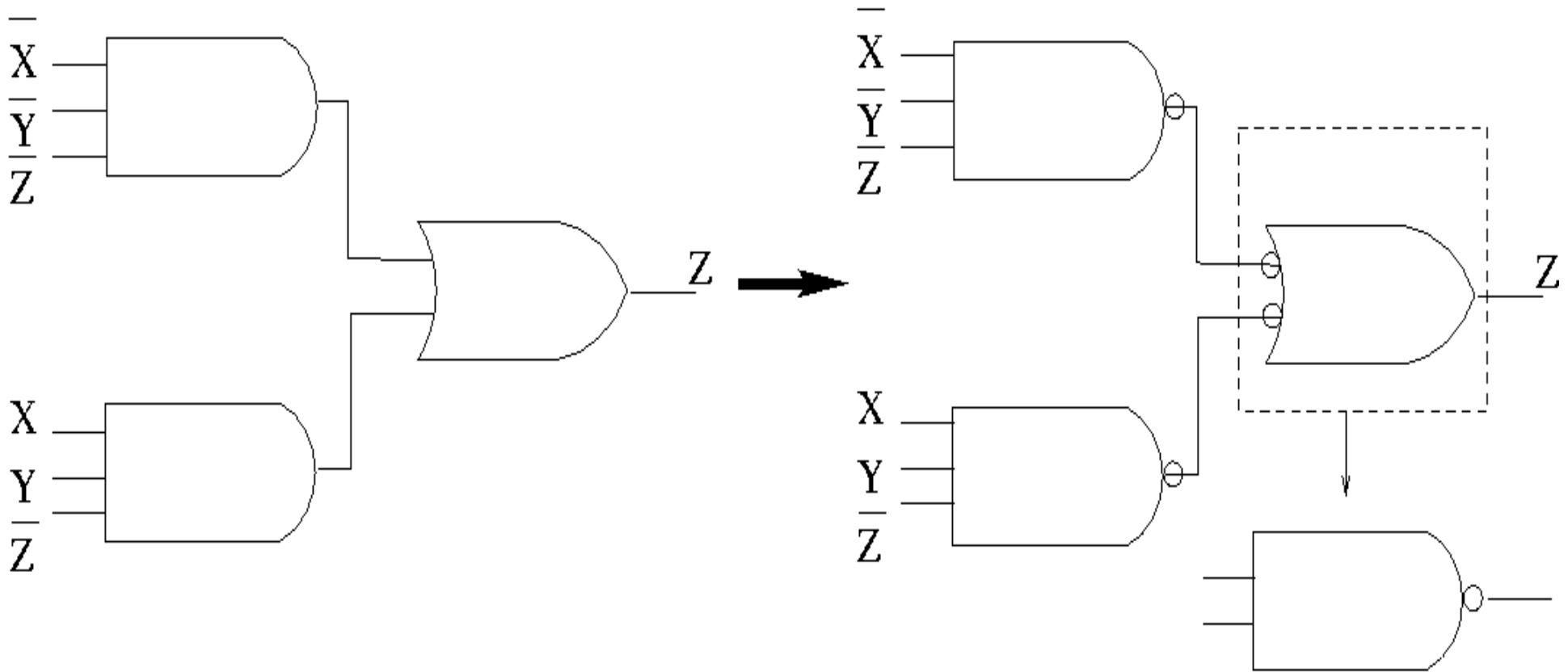
$$F(X,Y,Z) = \Sigma m(0,6)$$

1. Express F in SOP form:

$$F = X'Y'Z' + XYZ'$$

2. Obtain the AND-OR implementation for F .
3. Add bubbles and inverters to transform AND-OR to NAND-NAND gates.

Example (cont.)



Two-level implementation with NANDs

$$F = X'Y'Z' + XYZ'$$

Multilevel NAND Circuits

Starting from a multilevel circuit:

1. Convert all AND gates to NAND gates with AND-NOT graphic symbols.
2. Convert all OR gates to NAND gates with NOT-OR graphic symbols.
3. Check all the bubbles in the diagram. For every bubble that is not counteracted by another bubble along the same line, insert a NOT gate or complement the input literal from its original appearance.

Example

Use NAND gates and NOT gates to implement
 $Z = E'F(AB + C' + D') + GH$

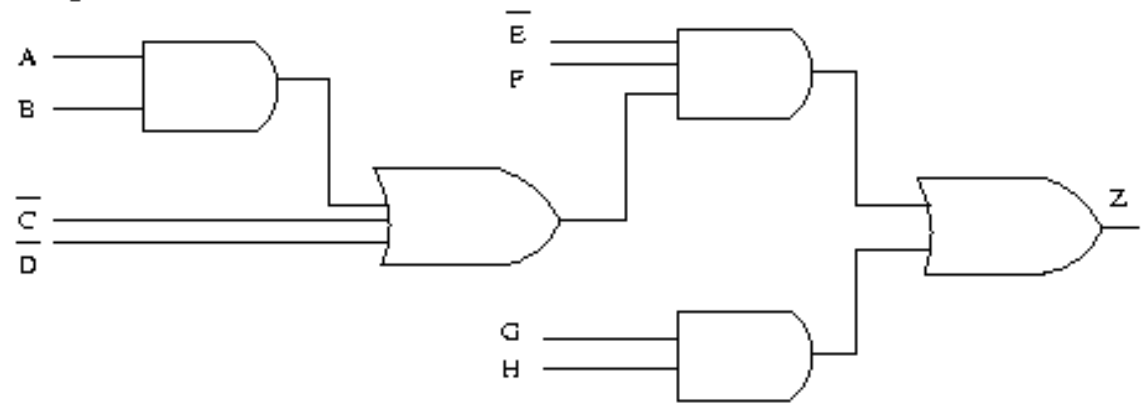
AB

$AB + C' + D'$

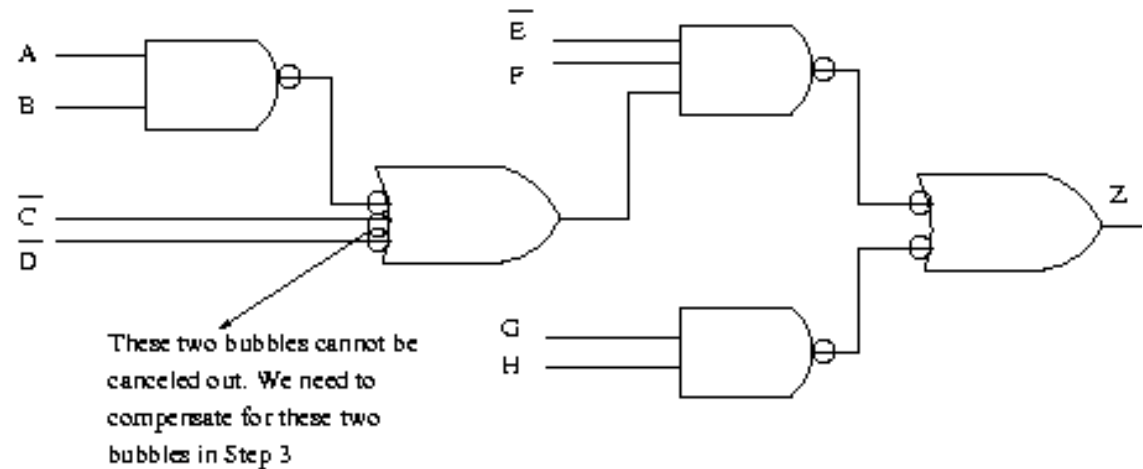
$E'F(AB + C' + D')$

$E'F(AB + C' + D') + GH$

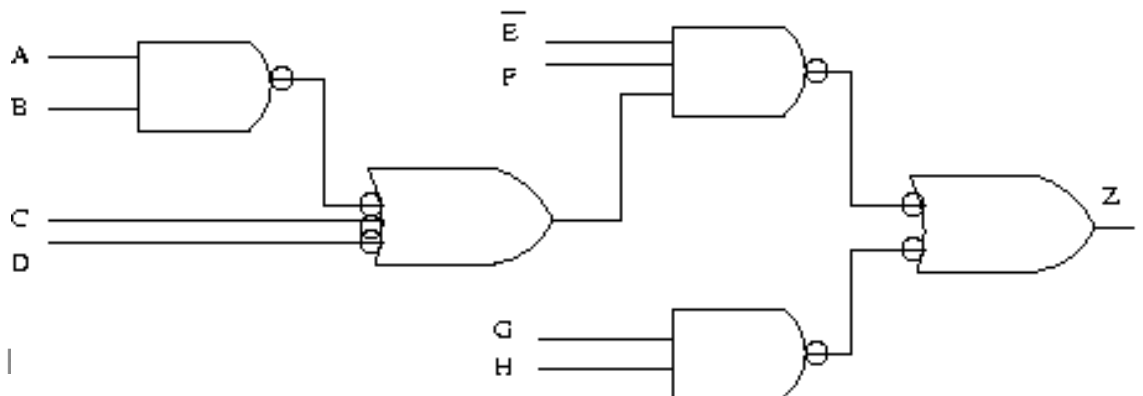
Step 1



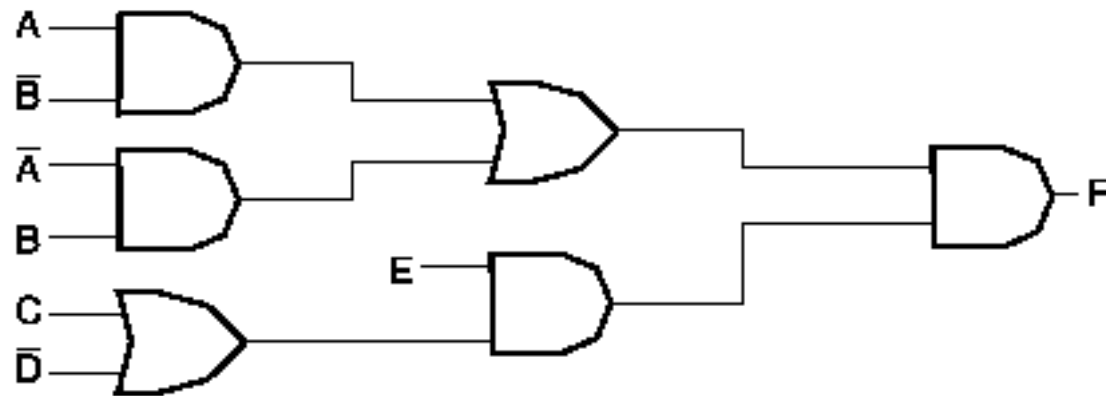
Step 2



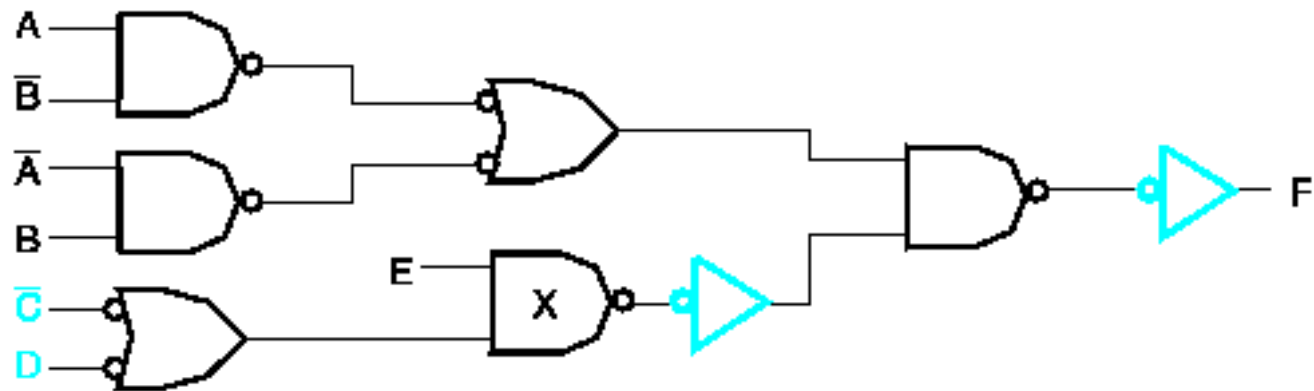
Step 3



Yet Another Example!



(a) AND – OR gates



(b) NAND gates

Fig. 2-32 Implementing $F = (A\bar{B} + \bar{A}B)E(C + \bar{D})$

Two-Level NOR Gate Implementation - Example

$$F(X,Y,Z) = \sum m(0,6)$$

1. Express F' in SOP form:

$$1. F' = \sum m(1,2,3,4,5,7)$$

$$= X'Y'Z + X'YZ' + X'YZ + XY'Z' + XY'Z + XYZ$$

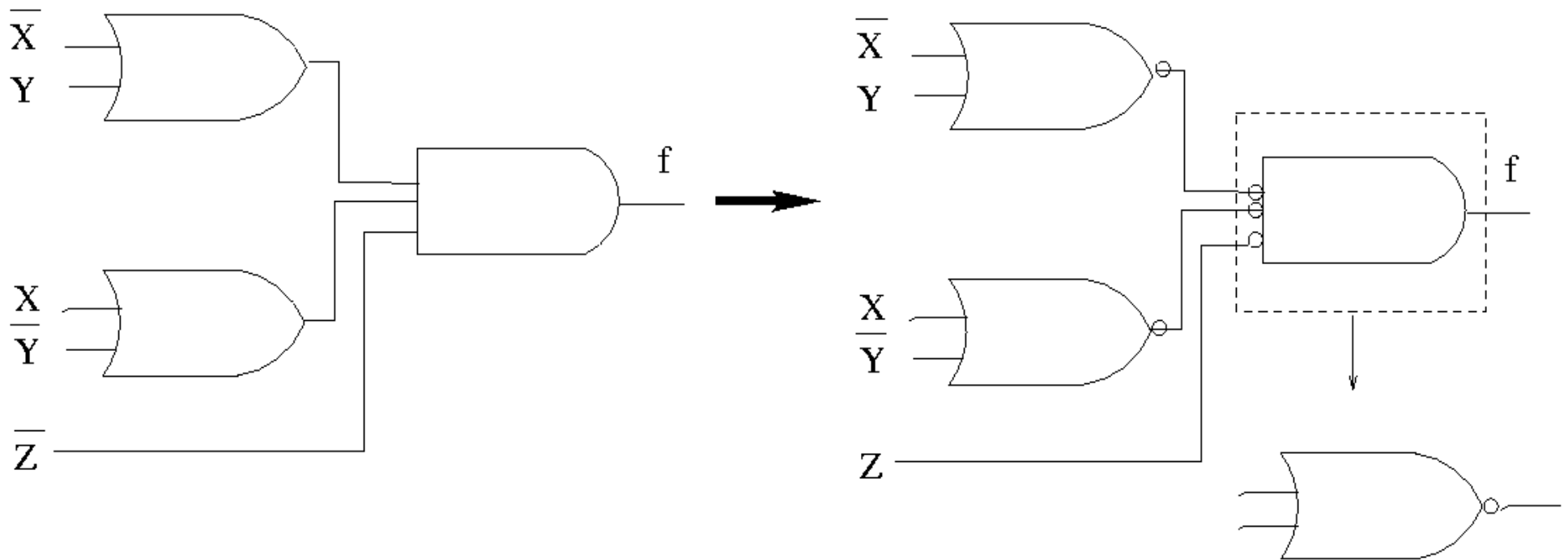
$$2. F' = XY' + X'Y + Z$$

2. Take the complement of F' to get F in the POS form: $F = (F')' = (X' + Y)(X + Y')Z'$

3. Obtain the OR-AND implementation for F .

4. Add bubbles and inverters to transform OR-AND implementation to NOR-NOR implementation.

Example (cont.)



Two-level implementation with NORs

$$F = (F')' = (X' + Y)(X + Y')Z'$$

Designing Combinational Logic Circuits (Procedure)

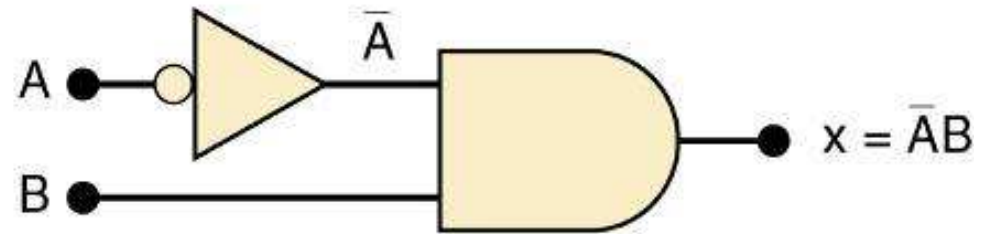
- To solve any logic design problem:
 1. Interpret the problem and set up its truth table.
 2. Write the AND (product) term for each case where output = 1.
 3. Combine the terms in SOP form.
 4. Simplify the output expression if possible.
 5. Implement the circuit for the final, simplified expression.

Designing Combinational Logic Circuits

Example - 1

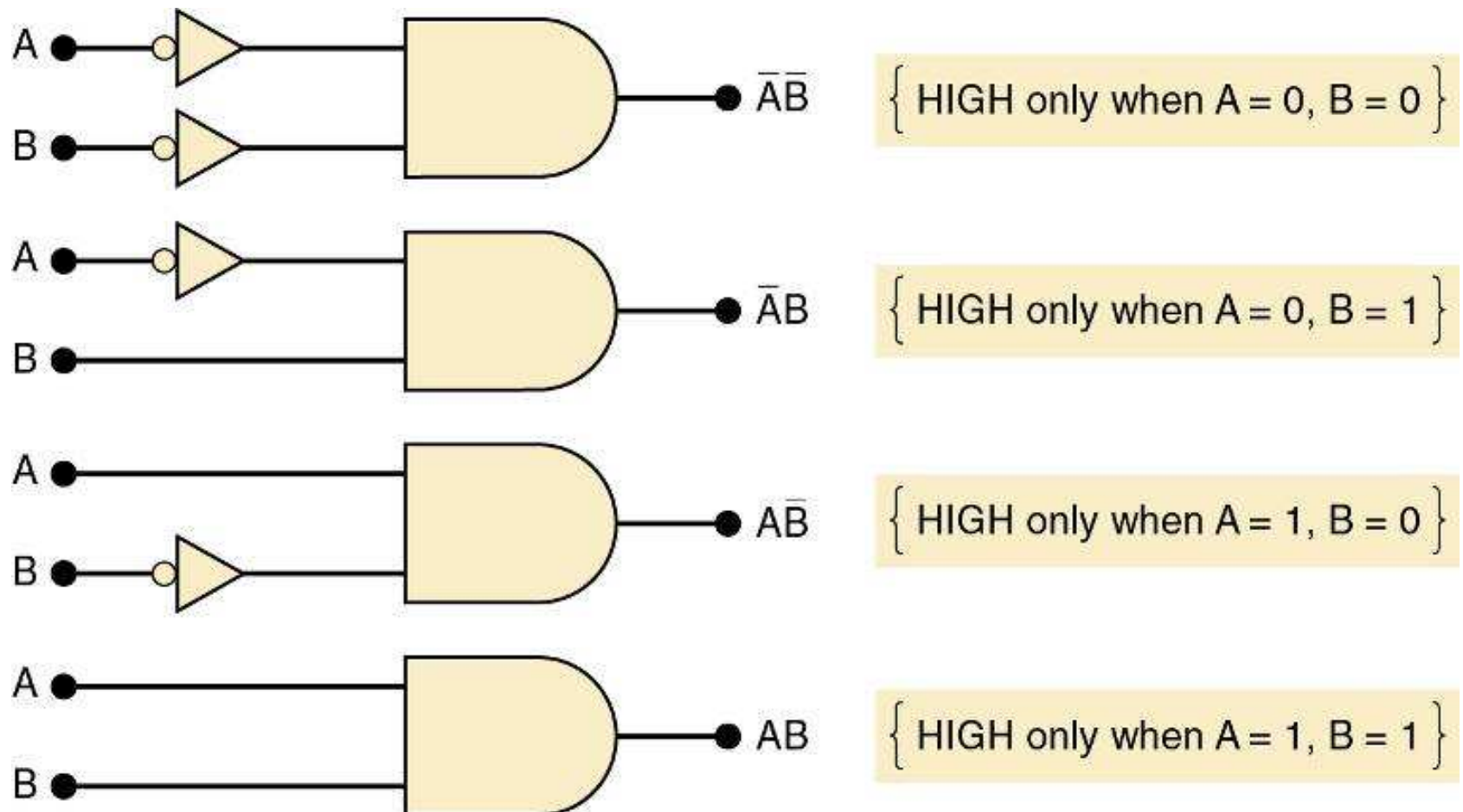
Circuit that produces a 1 output only for the $A = 0, B = 1$ condition.

A	B	x
0	0	0
0	1	1
1	0	0
1	1	0



Designing Combinational Logic Circuits

An **AND** gate with appropriate inputs can be used to produce a HIGH output for a specific set of input levels.

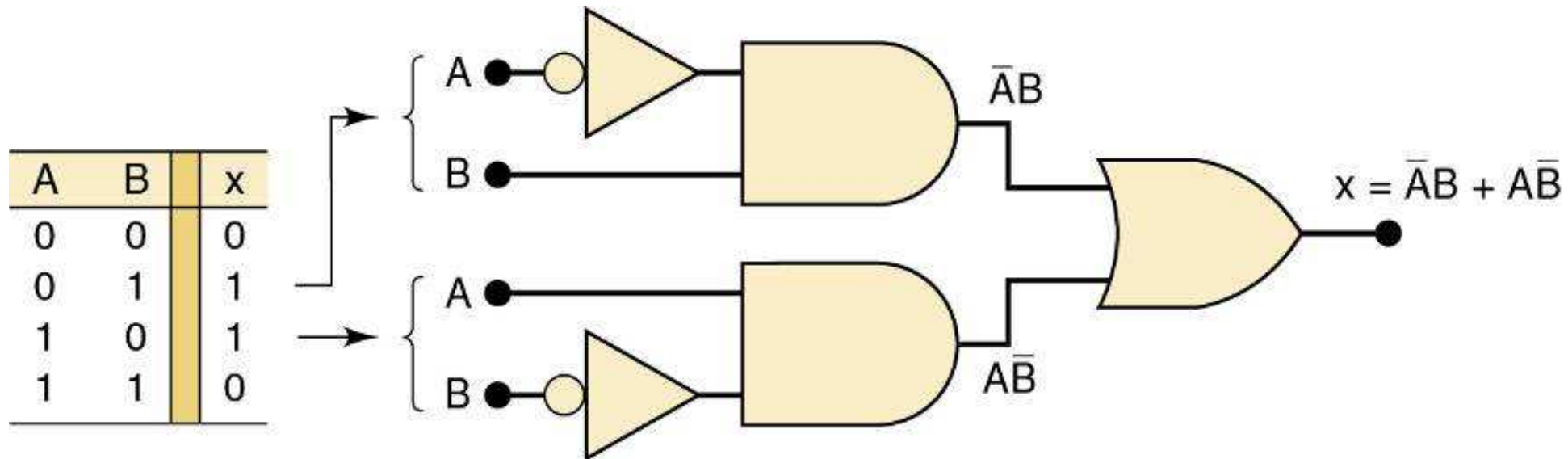


Designing Combinational Logic Circuits

Example - 2

Each set of input conditions that is to produce a 1 output is implemented by a separate **AND** gate.

The **AND** outputs are **OR**ed to produce the final output.



Designing Combinational Logic Circuits

Example - 3

Truth table for a 3-input circuit.

<i>A</i>	<i>B</i>	<i>C</i>	<i>x</i>
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

$\rightarrow \bar{A}\bar{B}\bar{C}$

$\rightarrow \bar{A}BC$

$\rightarrow ABC$

AND terms for each case where output is 1.

- **Now, add this terms**
- **Simplify the expression if possible**
- **Draw the circuit Diagram**

Designing Combinational Logic Circuits

Example - 4

Design a logic circuit with three inputs, A, B, and C. Output to be HIGH only when a majority inputs are HIGH.

Truth table.

<i>A</i>	<i>B</i>	<i>C</i>	<i>x</i>
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$\rightarrow \bar{A}BC$

$\rightarrow A\bar{B}C$

$\rightarrow AB\bar{C}$

$\rightarrow ABC$

AND terms for each case where output is 1.

SOP expression for the output:

$$x = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

Designing Combinational Logic Circuits

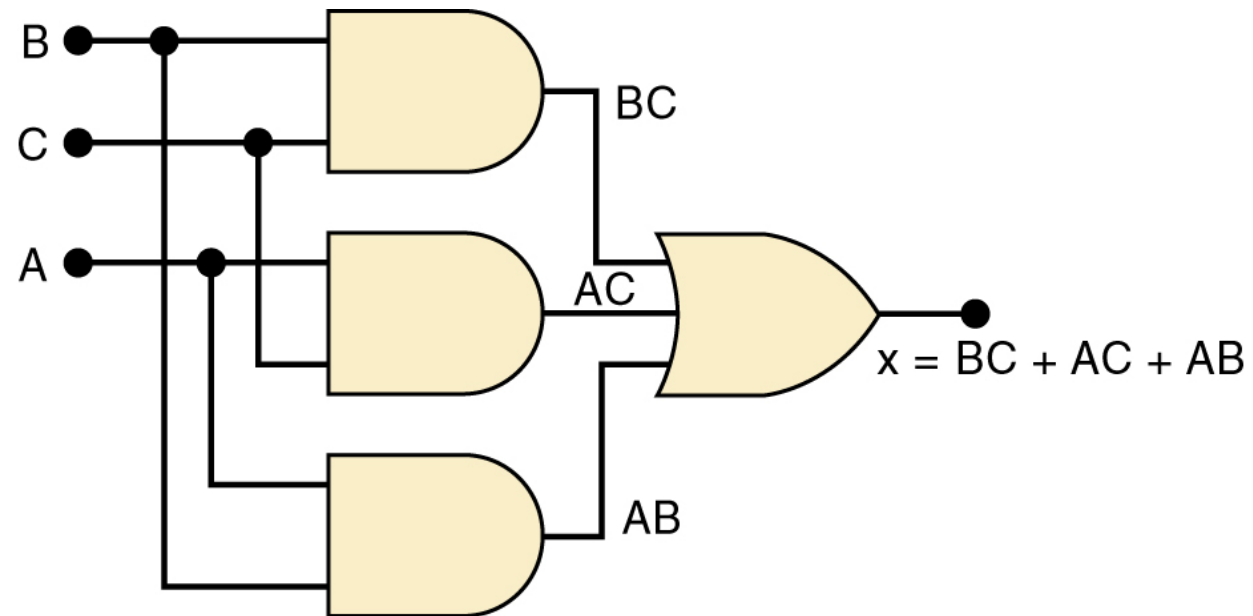
Design a logic circuit with three inputs, A, B, and C. Output to be HIGH only when a majority inputs are HIGH.

Simplified output expression:

$$x = BC + AC + AB$$

Implementing the circuit after factoring:

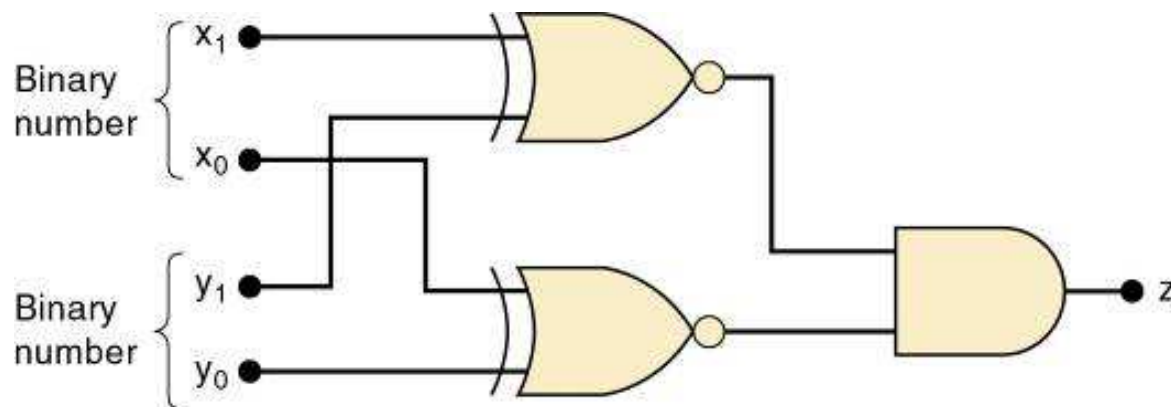
$$x = BC + AC + AB$$



Since the expression is in SOP form, the circuit is a group of **AND** gates, working into a single **OR** gate,

Example - 5 : Exclusive NOR Circuits

**Truth table and circuit
for detecting equality of
two-bit binary numbers.**



x_1	x_0	y_1	y_0	z (Output)
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Designing Combinational Logic Circuits

Example – 6 (Home Work)

- Design a logic circuit that is to produce a HIGH output when the voltage (represented by a four-bit binary number ABCD) is greater than 6V.

Designing Combinational Logic Circuits

Example – 7

- Four chairs A, B, C, and D are placed in a row. Each chair may be occupied (“1”) or empty (“0”). A Boolean function F is (“1”) if and only if there are two or more adjacent chairs that are empty.
 - a) Give the truth table defining the Boolean F .
 - b) Express F as a minterm expansion (standard sum of products).
 - c) Express F as a maxterm expansion (standard product of sums).
 - d) Simplify the minterm expansion of F .

Analysis procedure

To obtain the output Boolean functions from a logic diagram, proceed as follows:

1. Label all gate outputs that are a function of input variables with arbitrary symbols. Determine the Boolean functions for each gate output.
2. Label the gates that are a function of input variables and previously labeled gates with other arbitrary symbols. Find the Boolean functions for these gates.
3. Repeat the process outlined in step 2 until the outputs of the circuit are obtained.
4. By repeated substitution of previously defined functions, obtain the output Boolean functions in terms of input variables.

Example – 1

$$F_2 = AB + AC + BC; \quad T_1 = A + B + C; \quad T_2 = ABC; \quad T_3 = F_2' T_1;$$

$$F_1 = T_3 + T_2$$

$$F_1 = T_3 + T_2 = F_2' T_1 + ABC = A'BC' + A'B'C + AB'C' + ABC$$

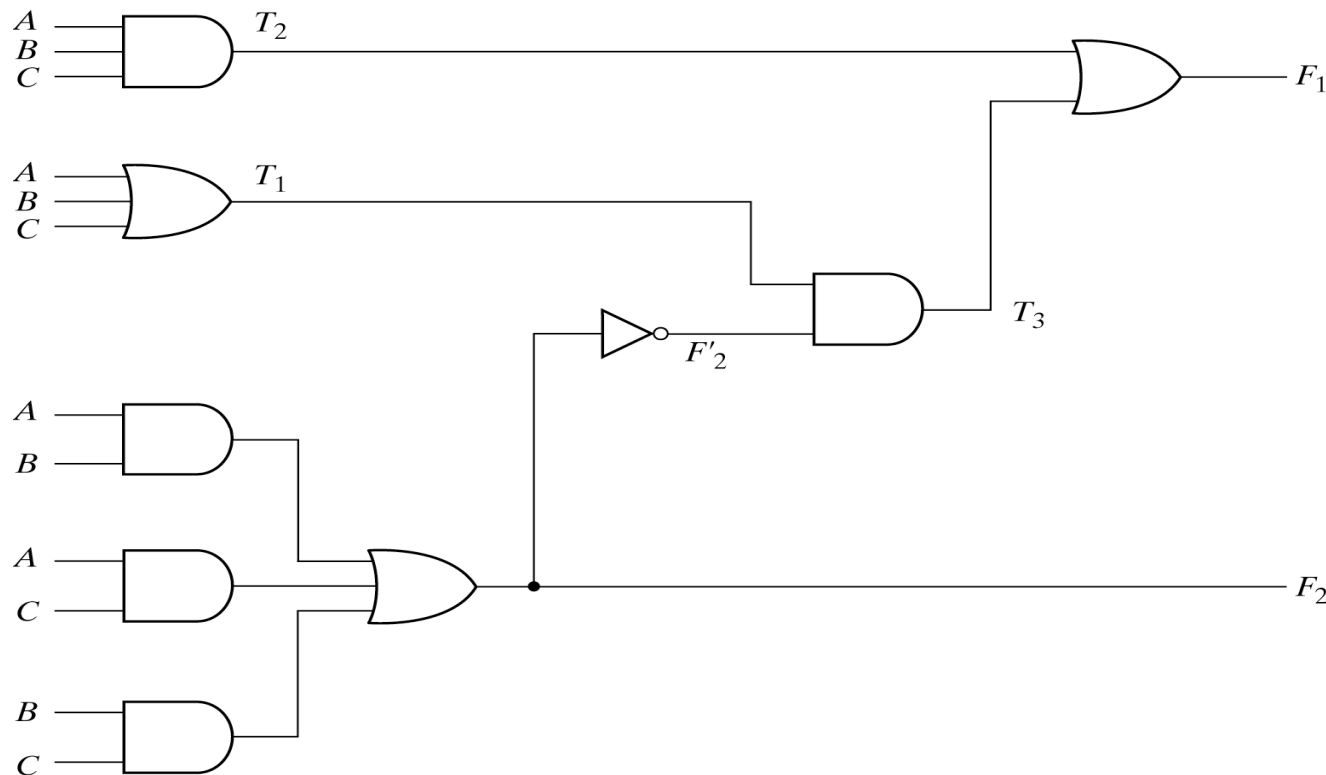


Fig. 4-2 Logic Diagram for Analysis Example

Derive truth table from logic diagram

- We can derive the truth table in Table 4-1 by using the circuit of Fig.4-2.

Table 4-1
Truth Table for the Logic Diagram of Fig. 4-2

<i>A</i>	<i>B</i>	<i>C</i>	<i>F</i> ₂	<i>F</i> ₂	<i>T</i> ₁	<i>T</i> ₂	<i>T</i> ₃	<i>F</i> ₁
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

Design procedure

1. Table 4-2 is a Code-Conversion example, first, we can list the relation of the BCD and Excess-3 codes in the truth table.

Table 4-2
Truth Table for Code-Conversion Example

Input BCD				Output Excess-3 Code			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Thank You

Next topic